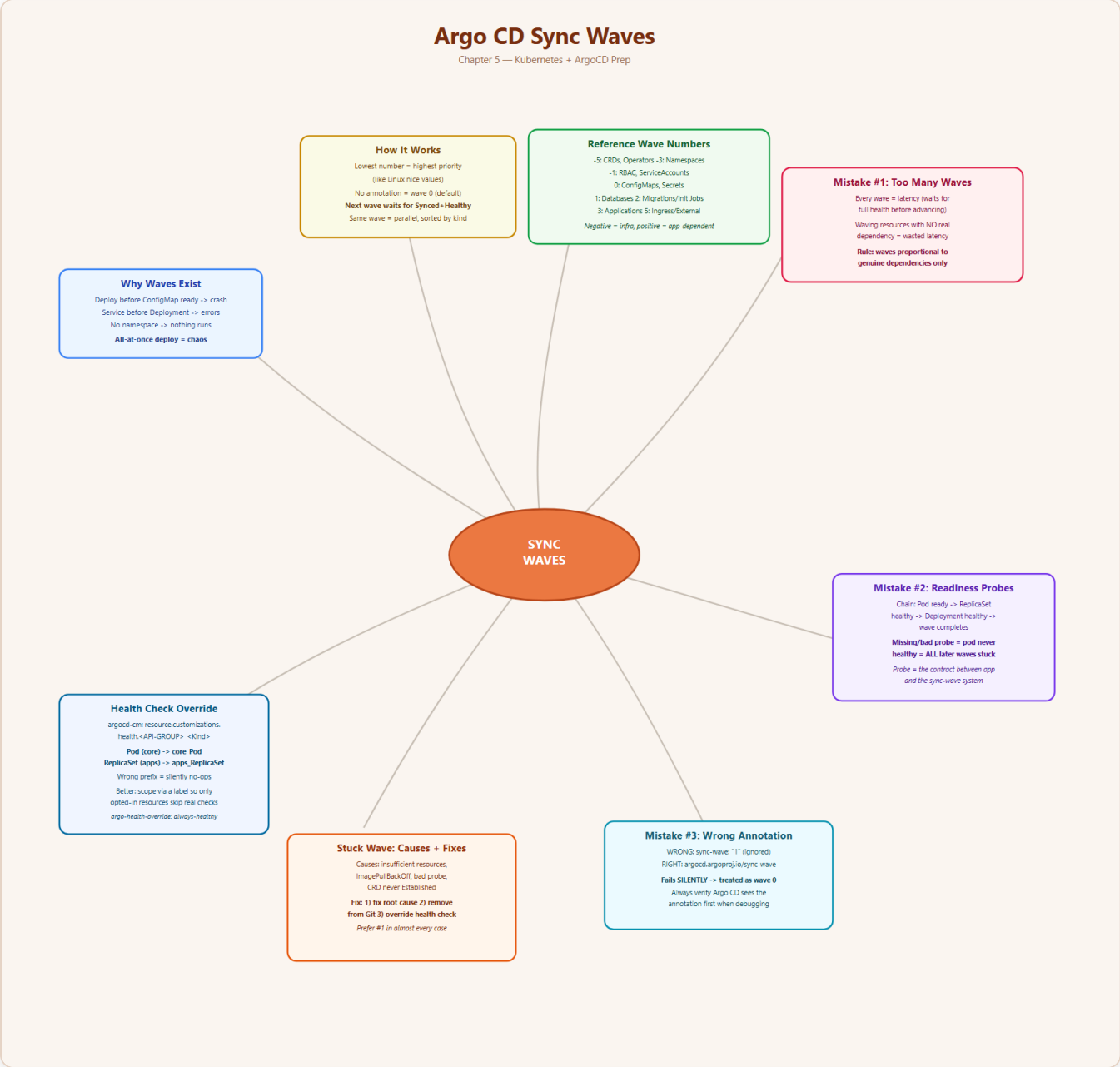
ARGO CD GITOPS › MY LEARNING NOTES › SESSION 05

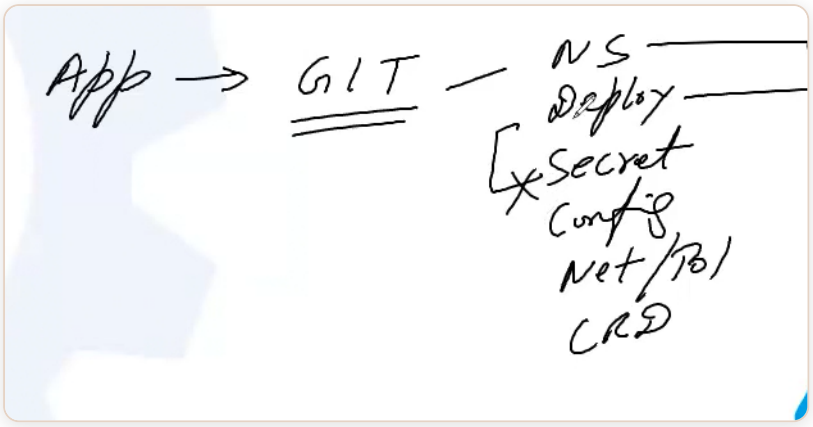
Argo CD Sync Waves

Topic: Deployment Ordering | Chapter 5 | 2026-09-13

Quick Recall — Mind Map



The Problem This Solves



All our application object files live in Git — namespace, deployment, secrets, config files, network policies, CRDs, everything. Kubernetes itself doesn't inherently care what order things show up in — but **applications have implicit ordering requirements** even if Kubernetes doesn't enforce them.

House-building analogy: you don't build the roof before the foundation. Foundation → pillars → walls → roof, in that order, or the whole thing doesn't stand. Deploying via GitOps has the exact same implicit-layering problem:

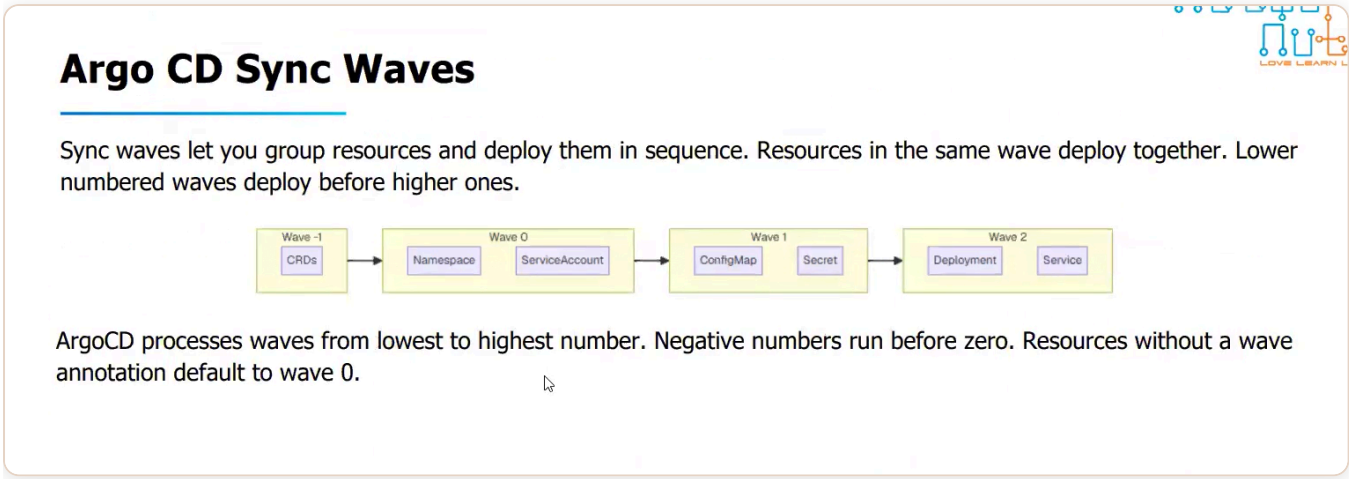
- Deployment starts, but its ConfigMap doesn't exist yet → **pod crashes**.
- Service is created, but the Deployment it targets doesn't exist yet → errors until the workload catches up.
- Pods can't run at all if their **namespace doesn't exist yet**.

> **The default is chaos**
>
> If Argo CD applies everything simultaneously (the default, with no sync waves), this is exactly the chaos that results. Sync waves let you define the Kubernetes object deployment sequence explicitly.

How Sync Waves Actually Work

- You assign a **wave number** to a resource via annotation. **Lowest number = highest priority** (deployed first) — same mental model as Linux `nice` values.
- **If no wave number is given, the resource defaults to wave `0`.**
- Argo CD deploys wave **-5 → -1 → 0 → 1 → 2 → 3 …** in strict numerical order, lowest first.
- **Argo CD will not start the next wave until every resource in the current wave is both Synced AND Healthy.** This is what actually enforces the ordering.
- Resources **within the same wave number** deploy together (sorted by kind, then alphabetically by name).

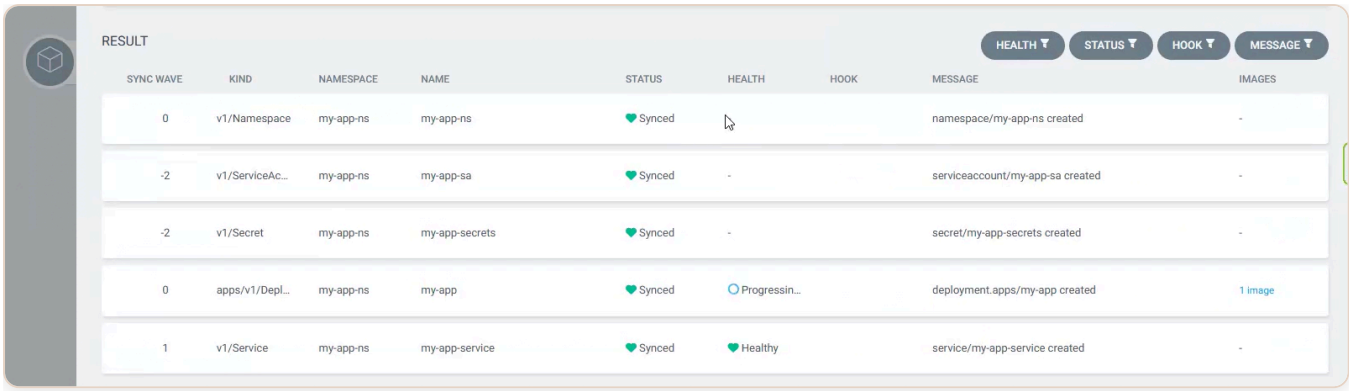
Worked Example — Assigning Waves to a Real App



Wave	Resources	Why
-1	CRD	Most fundamental — nothing else can exist without it
0	Namespace, ServiceAccount	Parallel-safe, no dependency between them
1	ConfigMap, Secret	Configuration layer, needed before workloads start
2	Deployment, Service	Application layer — depends on everything above

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: my-app-ns
  annotations:
    argocd.argoproj.io/sync-wave: "-5"
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-app-sa
  namespace: my-app-ns
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
```

Confirmed live in the UI's sync result view — the `SYNC WAVE` column showed exactly this order: `ServiceAccount` (-2) and `Secret` (-2) together, then `Namespace` (0), `Deployment` (0), then `Service` (1) last:



General Ordering Convention

Wave range	Purpose
-5 to -1	Infrastructure (namespaces, CRDs)
0 to 2	Configuration (configs, secrets)
3 to 5	Applications (deployments, services)
6+	External resources (ingress, monitoring)

Wave	Purpose
-5	CRDs, Operators
-3	Namespaces
-1	RBAC, ServiceAccounts
0	ConfigMaps, Secrets
1	Databases, Caches
2	Migrations, Init Jobs
3	Applications
5	Ingress, External Services

Checking Sync Wave Status

BASH

```bash
# Show the current operation details, including resources being synced
argocd app get networknuts-app --show-operation

# List resources in the application (with their wave numbers)
argocd app resources networknuts-app --output tree=detailed
```

⚠ Common Mistake #1 — Too Many Waves

Argo CD **waits for a wave to be fully healthy before starting the next one** — every wave adds real latency. Teams that add a sync-wave annotation to *every* resource "just in case" end up with ten-wave configurations where waves 3, 4, and 5 contain resources with **no actual dependency** between them.

> **Rule**
>
> Keep the number of waves proportional to the number of genuine dependencies. Don't wave things that don't actually need to wait on each other.

⚠ Common Mistake #2 — Missing or Incorrect Readiness Probes

The trace-through: Deployment creates ReplicaSet, ReplicaSet creates Pod. Deployment is healthy only if ReplicaSet is healthy, which is healthy only if the Pod is healthy, which is healthy only if its containers pass readiness. **If the readiness probe is missing or misconfigured, the pod never reports healthy — so the entire wave never completes — so every subsequent wave stays blocked indefinitely.**

> **The probe is the contract**
>
> The readiness probe is the literal contract between your application and the sync wave system. If it doesn't accurately reflect real readiness, the next wave starts too early or never starts at all.

Sync Waves vs. Kubernetes Probes

Kubernetes probes check whether a *container* is ready; Argo CD sync waves control the *order* resources deploy in. Together: a wave only finishes once its resources pass their probe-driven health checks. Wave 0 must reach fully `Healthy` before wave 1 even starts.

⚠ Common Mistake #3 — The Wrong Annotation Key (Silent Failure)

The first step in debugging any sync-wave problem: confirm Argo CD actually sees your annotation. A typo in the annotation key means Argo CD silently treats the resource as wave `0` — no error, no warning.

```yaml
# WRONG - missing the full annotation key, Argo CD ignores this entirely
metadata:
  annotations:
    sync-wave: "1"

# CORRECT
metadata:
  annotations:
    argocd.argoproj.io/sync-wave: "1"
```

Other Reasons a Sync Wave Gets Stuck

- **Insufficient resources** — pod can't be scheduled; Deployment stays in `Progressing` forever.
- **ImagePullBackOff** — image doesn't exist or credentials are wrong; Deployment never becomes healthy.
- **Readiness probe failure** — pod starts, probe never passes; Deployment stuck `Progressing`.
- **CRD not becoming** `Established` — schema validation errors; the wave containing that CRD stays stuck.

Fixing a Stuck Sync Wave — 3 Options

1. **Fix the stuck resource** (right answer in most cases) — find and fix why it's unhealthy.
2. **Remove the stuck resource from the application** — if optional, remove from Git and sync again; Argo CD prunes it.
3. **Override the health check for that resource type** — treat certain resource types as always-healthy via the Argo CD ConfigMap.

Overriding Health Checks via argocd-cm

BASH

```bash
kubectl -n argocd get cm
kubectl edit configmap argocd-cm -n argocd
```

YAML

```yaml
kind: ConfigMap
metadata:
  name: argocd-cm
  namespace: argocd
data:
  # Example for a core Pod resource treated as always healthy
  resource.customizations.health.core_Pod: |
    hs = {}
    hs.status = "Healthy"
    hs.message = "Forced to be always healthy"
    return hs
```

Why core_Pod but apps_ReplicaSet? The API group prefix matters

The key format is `resource.customizations.health.<API-GROUP>_<Kind>`. Check via `kubectl api-resources`: `Pod` is core API group (bare `v1`) → `core_Pod`; `ReplicaSet` is the `apps` API group (`apps/v1`) → `apps_ReplicaSet`. Get this wrong and the override silently does nothing — same failure mode as the wrong annotation key.

A Smarter Override — Only Force-Healthy Resources That Opt In via a Label

```yaml
resource.customizations.health.apps_ReplicaSet: |
  -- Check if the specific label exists on the ReplicaSet
  if obj.metadata.labels ~= nil and obj.metadata.labels["argo-health-override"] == "always-
healthy" then
    hs.status = "Healthy"
    hs.message = "Forced to Healthy via application-specific label override"
    return hs
  end

  -- Fallback to standard ArgoCD ReplicaSet health logic if label is missing
  if obj.status ~= nil and obj.status.replicas ~= nil then
    if obj.spec.replicas == obj.status.readyReplicas then
      hs.status = "Healthy"
      return hs
    end
  end
  hs.status = "Progressing"
  return hs
```

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: my-replicaset
  labels:
    argo-health-override: "always-healthy"   # triggers the custom Lua condition above
```

This way, only ReplicaSets explicitly opting in via that label skip real health evaluation — every other ReplicaSet in the cluster still gets genuine health checking.

What I Practiced

- [ ] Built a real multi-resource app with explicit `argocd.argoproj.io/sync-wave` annotations and watched the UI's sync result table confirm resources landed in the exact wave order assigned.
- [ ] Deliberately used the wrong annotation key and confirmed Argo CD silently treated that resource as wave 0.
- [ ] Reasoned through the full health-dependency chain that makes a missing readiness probe block an entire wave indefinitely.
- [ ] Used `argocd app get --show-operation` and `argocd app resources --output tree=detailed` to inspect wave/sync status from the CLI.
- [ ] Overrode health checks in `argocd-cm` for `core_Pod` and understood the API-group-prefix requirement.
- [ ] Wrote a scoped Lua health-check override that only forces "Healthy" for resources carrying a specific label.

Merged into the master reference notes: argocd-master-notes.md — new "5. Sync Waves" section.